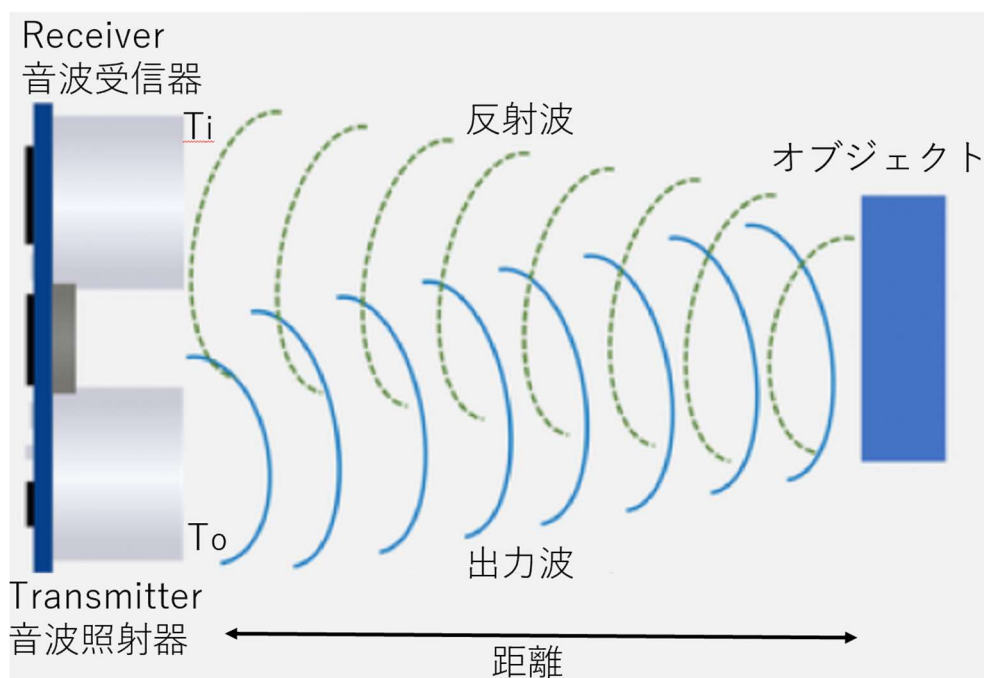
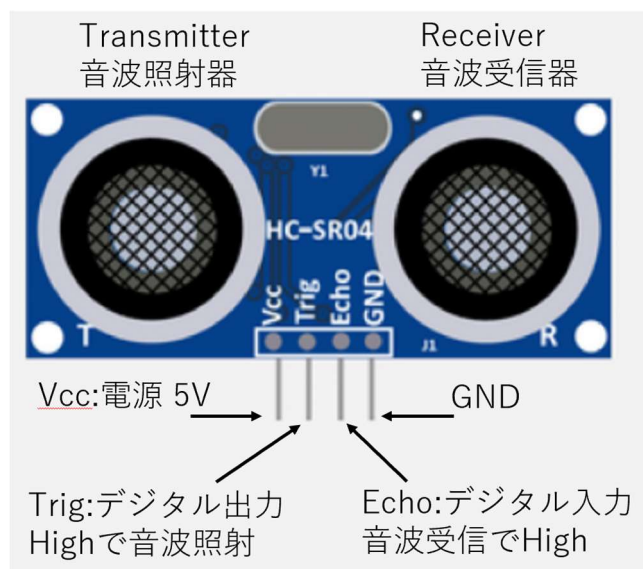


1 超音波距離センサ

1-1 ピンの概説と測定原理



$$\text{距離 [m]} = \text{音速 } c \text{ [m/sec]} \times \text{時間 } t \text{ [sec]}$$

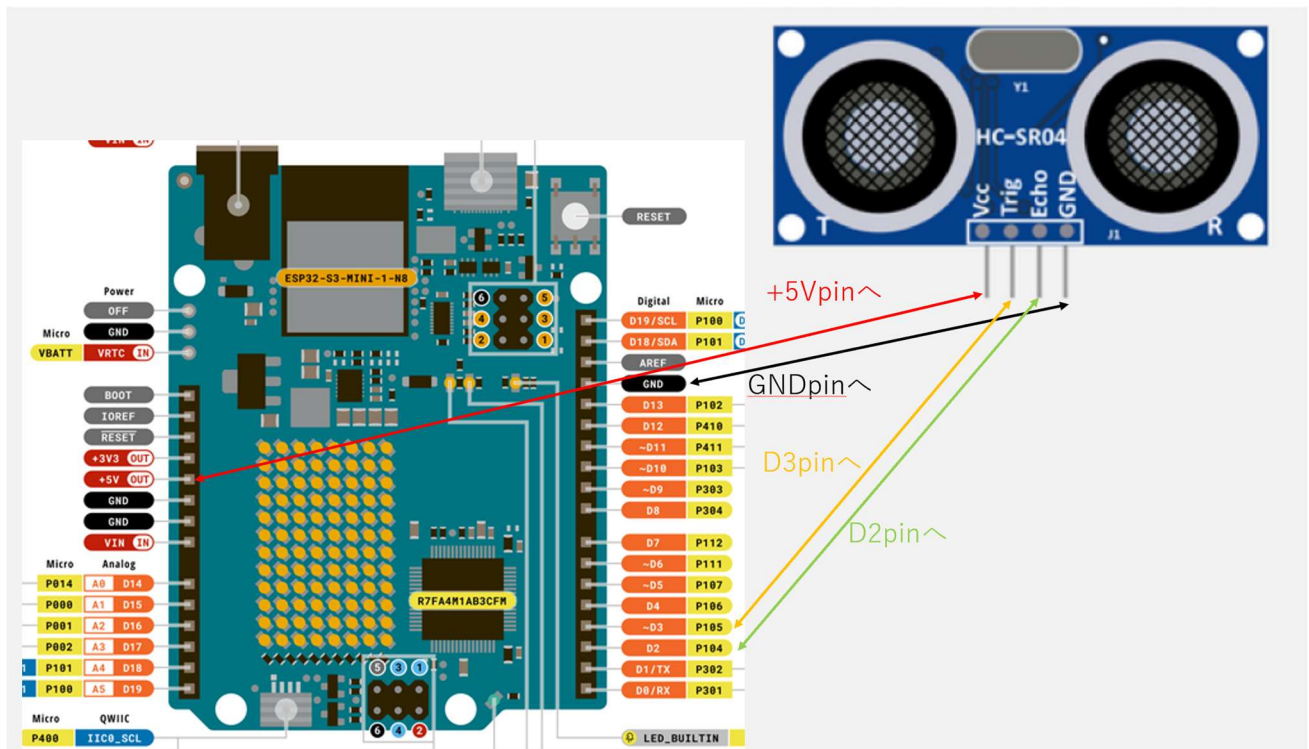
$$\text{音速 } c \text{ [m/sec]} = 331.4 \times 0.6 \times \text{気体温度 } K \text{ [}^{\circ}\text{C]}$$

$$\text{測定時間 } T_m \text{ [sec]} = \text{受信時刻 } T_i \text{ [sec]} - \text{照射時刻 } T_o \text{ [sec]}$$

測定時間は往復時間なので1/2とする

$$\text{時間 } t \text{ [sec]} = 0.5 \times \text{測定時間 } T_m \text{ [sec]}$$

1-2 超音波センサと Arduino の配線例



1-3 プログラム

1-3-1 ピンなどの定義

```
1  int Trig = 3; //trigピン
2  int Echo = 2; //Echoピン
3
4  double time = 0.0; //測定時間
5  double dist = 0.0; //距離
6  double temp = 24.0 //気温
7
8  void setup() {
9      Serial.begin(9600);
10     pinMode(Trig,OUTPUT); //音波出力ピン OUTPUTとする
11     pinMode(Echo,INPUT); //音波受信ピン INPUTとする
12 }
```

1-3-2 距離の測定

```
14 void loop() {
15     //音波出力
16     //LOWで照射止めて待機, HIGHで照射
17     //待機→1照射→待機の繰り返し.
18     digitalWrite(Trig,LOW);
19     delayMicroseconds(2);
20     digitalWrite(Trig,HIGH);
21     delayMicroseconds(10);
22     digitalWrite(Trig,LOW);
23
24     //音波受信
25     time = pulseIn(Echo,HIGH); //照射から受信までの時間を計測
26     if(time>0){
27         time = 0.5*time;
28         dist = (331.4 + 0.6 * temp)*time/1000000; //マイクロ秒を秒に変換
29         Serial.print("Distance = ");
30         Serial.print(dist);
31         Serial.print("m");
32         //mだと値が小さく分かりづらい場合は100倍してcmもしくは1000倍してmmとしてもよい
33     }
34 }
35 }
```

2 サーボモータ

2-1 サーボモータの概要

モータのほかに制御機構などが組み込まれたアクチュエータであり, 指定した信号 (pwm もしくはメーカー毎に定義された文字列コード) を受け取るとその信号に応じて指定した角度に回転する.

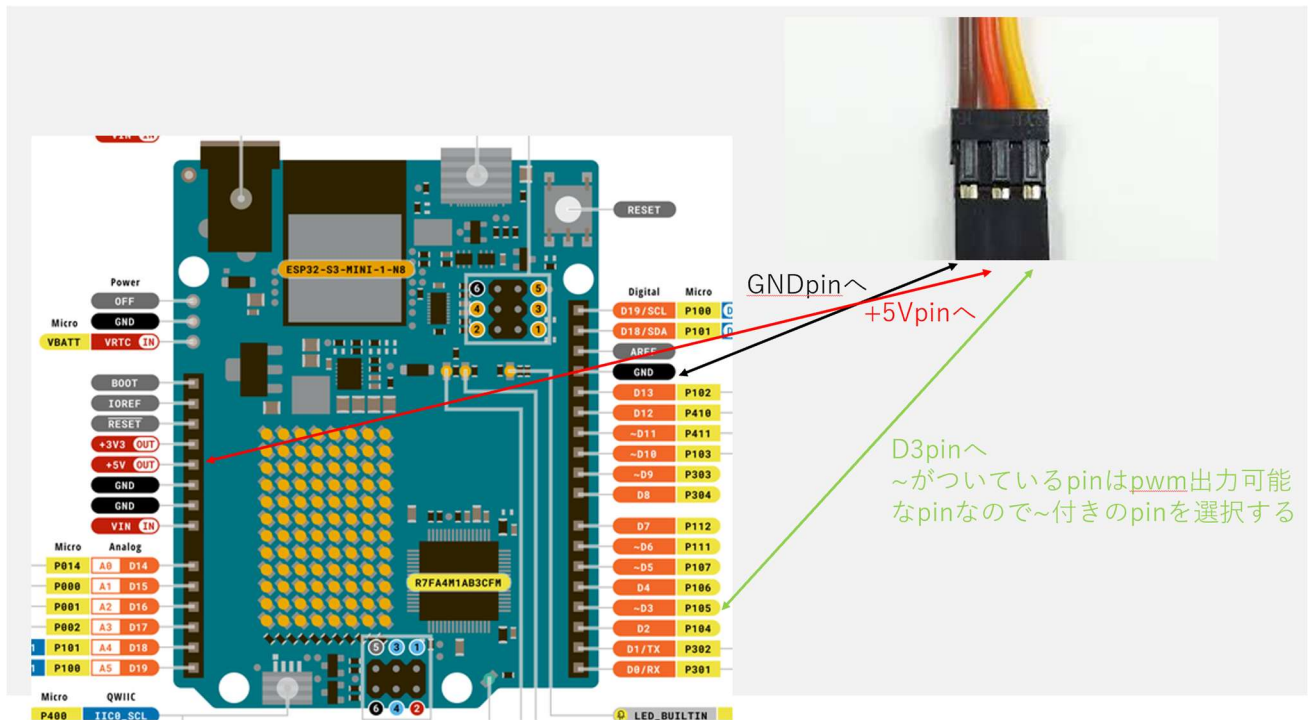
一般的にトルクもそこそこあり (今回用いるサーボは $6.5\text{kgf}\cdot\text{cm}$), ロボットの関節や何らかの対象の向きを制御するようなシステムによく用いられる.

今回用いるサーボは pwm 信号の duty 比に応じて角度を制御し, サーボ本体にエンコーダが無いためそのままでは現在角度のフィードバックは得られないが構造が単純で安価に入手できる.



GND Vcc:5V Sig:デジタル入力
PWM信号のduty比で角度制御

2-2 サーボモータと Arduino の配線例



2-3 プログラム

2-3-1 SG-5010 の pwm による制御

サーボはものによるが DC モータのように無限回転しないものもあるので、事前によく確認すること。
今回用いる SG-5010 は 0 deg から 180 deg の範囲で回転する。
データシートによると今回用いるサーボである SG-5010 の pwm 周りの制御仕様は以下のとおり
(なお, pwm 信号についてはロボット設計工学を復習参照すること)

Pwm の周期 : 20 ミリ秒

500 マイクロ秒で 0 deg → 500 マイクロ秒/20 ミリ秒なので, duty 比は 2.5%

2400 マイクロ秒で 180 deg → 2400 マイクロ秒/20 ミリ秒なので, duty 比は 12%

このとき求める必要があるものは任意の角度とするためにはどのくらいの秒数 pwm を on の状態にすればよいのかということ。つまり, 任意の角度とするための duty 比を求める必要がある。

まず, 任意の角度とするための on の秒数を求める。

全体の時間幅は 2400 マイクロ秒 - 500 マイクロ秒 = 1900 マイクロ秒

1deg 当たりの on の秒数はこれに最大回転範囲における 1deg の割合をかければよいので

$$\frac{\text{Angle}}{180} \times (2400 - 500)$$

0deg における秒数は 500 マイクロ秒なので任意の角度の on の秒数 T_{on} は

$$T_{on} = \frac{\text{Angle}}{180} \times (2400 - 500) + 500$$

Pwm のサイクルは 20 ミリ秒であるから duty は 20 ミリ秒の範囲における割合を求めればよい。
ただし、Ton はマイクロ秒であるので、ミリ秒に合わせるために 1000 で割ることも忘れずに。
また 100 分率（パーセント）にするために 100 倍する。

$$duty = \frac{T_{on}}{20} \times \frac{1}{1000} \times 100$$

以上より duty 比をもとめてデジタル信号としてサーボのシグナル線に arduino から出力すればよいわけであるが、上記のような計算をいちいち記述しなくとも arduino にはサーボモータ用の制御ライブラリが用意されている。

（ライブラリを用いれば簡単にコードが記述できるが、使用する際は動作原理など何をやっているのか中身をきちんと理解してから使用することが大事）

arduino のサーボ制御ライブラリとして Servo.h をインクルードする。

Servo 型変数として ser1 を定義し、pwm 信号を出力する pin として D3 を定義する（～付きならどこでもよい）

setup 関数にて Servo.h の attach 関数を呼び出して、信号線の pin の設定と on 時間の範囲を定義する。
その後、D3pin の動作を出力ピンとして設定しておく。

```
1  #include "Servo.h"
2
3  Servo ser1; //サーボモータ制御用変数
4  int pin1 = 3; //pwm出力可能なpinとしてD3を使用
5
6  void setup() {
7      //pinの設定とon時間の範囲を設定
8      ser1.attach(pin1, 500, 2400);
9      //D3pinを出力ピンに
10     pinMode(pin1, OUTPUT);
11 }
```

Loop 関数にて、Servo.h の write 関数を用いることで指定した角度に回転制御することができる。

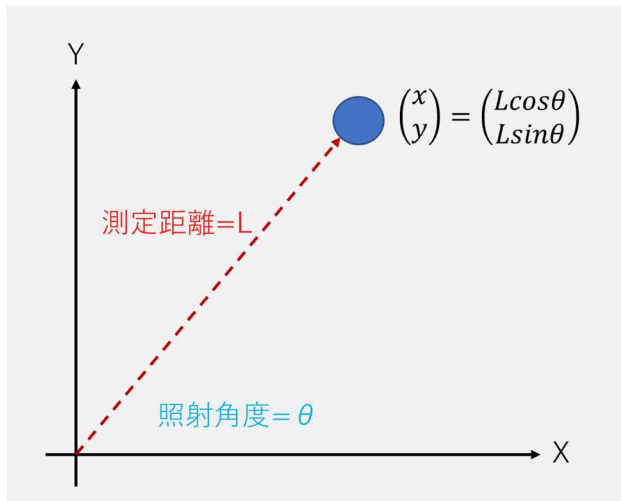
```
13 void loop() {
14     //Servo.hのwrite関数に指定の角度[deg]を入力することで角度制御可
15     ser1.write(0);
16     delay(1000);
17     ser1.write(90);
18     delay(1000);
19     ser1.write(170);
20     delay(1000);
21 }
```


3 応用

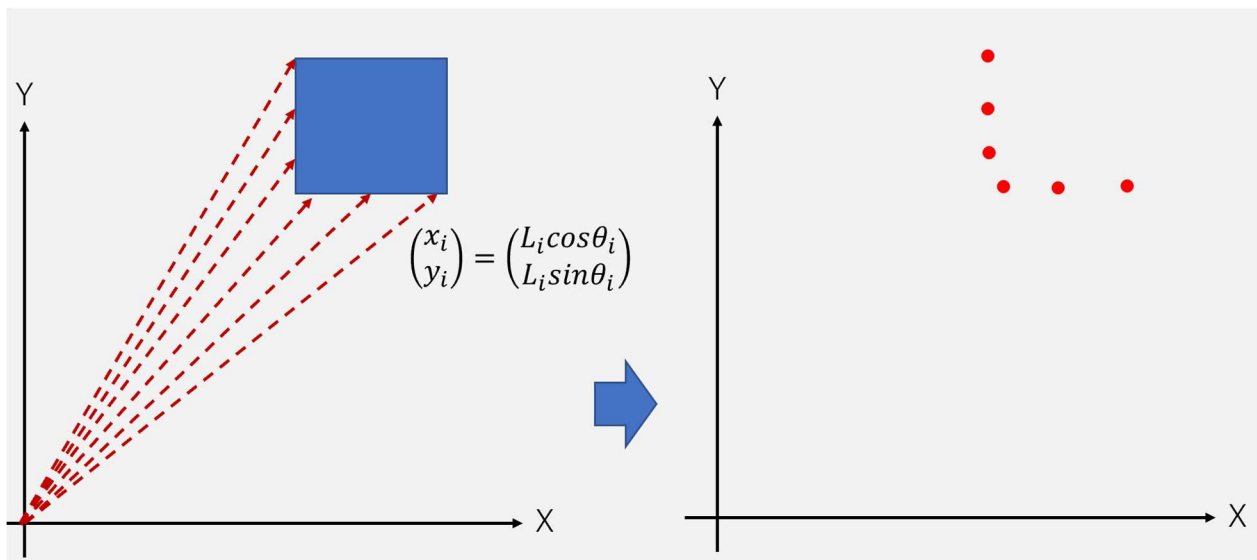
3-1 測定データによる環境形状の可視化

超音波センサはセンサ正面に位置する物体までの距離を測定することが可能であるが、取得できる値はその距離のみである。

しかしながらセンサが向いている角度，すなわち音波の照射角度が分かれば以下のように極座標表現により対象の位置情報を計算することができる。



以上を 1deg 刻みに計算して、測定した位置情報を束ねることである種の地図（見取り図）のような情報を作成することができる。（ただし上記の座標系はセンサに固定なので注意。基準座標系ではないので全体の地図を作る場合は別途、基準座標系におけるセンサの位置姿勢を用いて座標変換する必要がある。）



よってサーボモータと超音波センサを組み合わせることで上記のような環境計測が可能となる。

今回用いるサーボはエンコーダを持たず現在角度は読み取れないが、指定角度に制御することは可能であるので、指定した角度を何等かの形で保持しておけば、角度制御後に測定した距離情報と組み合わせることで上記のような計算ができるはずである。各自で挑戦してみよう。（ただし、サーボホーンに超音波センサを取り付けるパーツを自作する必要がある。）

3-2 グラフ表示

Arduino はそれ単体では計算結果を可視化することは LED マトリックスを駆使すれば不可能ではないが、困難である。

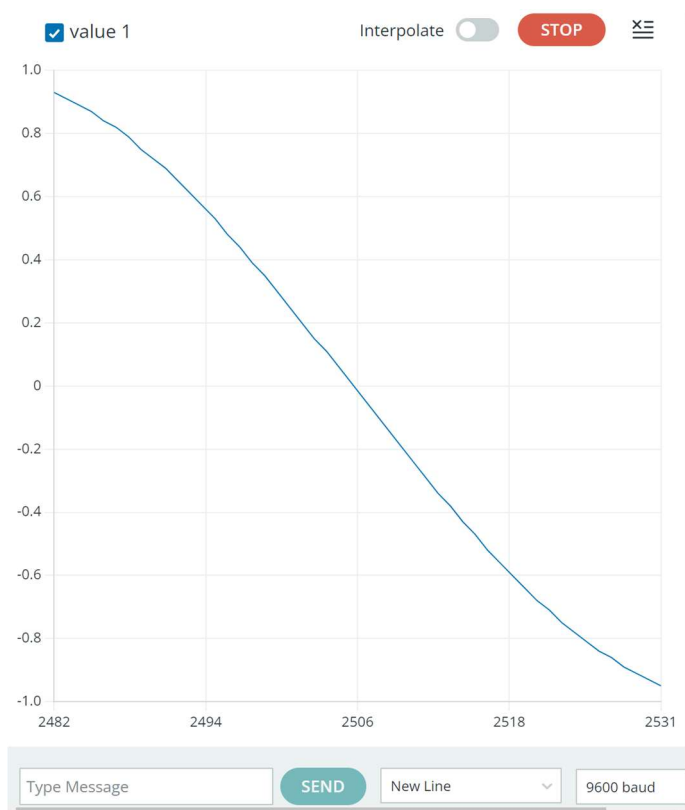
しかしながら、Arduino IDE には plotter という可視化ツールが用意されている。ここでは簡単なコードでグラフを表示することを試してみる。

以下のように sin 波の値を出力するコードを作成する。

```
1  float t = 0;
2
3  void setup()
4  {
5      Serial.begin(9600);
6  }
7
8  void loop() {
9      t += 0.05;
10     if(t >= 360) t = 0;
11     float p = sin(t);
12
13     String str = String(p);
14     Serial.println(str);
15 }
```

Arduino に書き込み後、IDE の tools から Serial plotter を選択する。

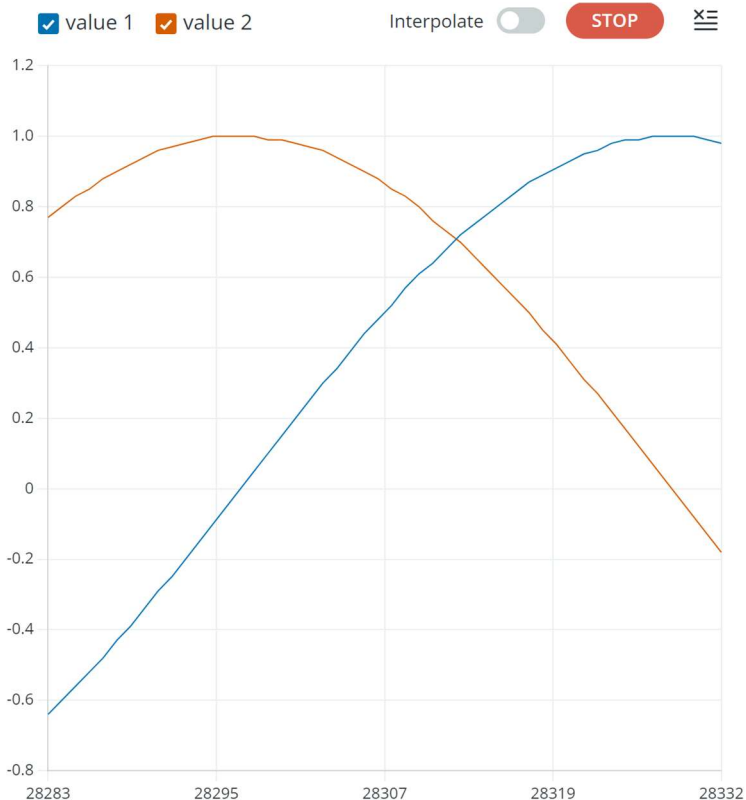
すると図のようにグラフが表示される。



2つ以上のグラフを表示させる場合は","で区切ることで表示できる。
以下のように cos の値を計算するコードを追記する。

```
1  float t = 0;
2
3  void setup()
4  {
5      Serial.begin(9600);
6  }
7
8  void loop() {
9      t += 0.05;
10     if(t >=360)t=0;
11     float p = sin(t) ;
12     float q = cos(t);
13
14     String str1 = String(p);
15     String str2 = String(q);
16     Serial.print(str1);
17     Serial.print(",");
18     Serial.print(str2);
19     Serial.println("");
20 }
```

実行すると以下のように2つグラフが表示される。



このように plotter を用いると簡単にグラフ表示できる。しかし横軸は loop 関数の処理毎に勝手に増大するため、3-1 のグラフは表示できない。この場合外部にデータを記録してグラフ化する必要がある。